BÁO CÁO BÀI THỰC HÀNH LAB 01

SQL trên Google Cloud Shell với SQLite

Môn: Introduction to Data Science — Học viện Công nghệ Bưu chính Viễn thông
Sinh viên: Nguyễn Danh Thái MSSV: B23DCKH105

---

1. Thông tin chung

Bài thực hành xây dựng cơ sở dữ liệu bán hàng trực tuyến gồm bốn bảng trên SQLite, thực hiện CRUD, truy vấn nhiều bảng, tổng hợp, truy vấn con, hàm cửa sổ, rồi tạo chỉ mục và phân tích kế hoạch thực thi. Toàn bộ 21 yêu cầu S01–S21 đã cài đặt và chạy kiểm chứng.

- Môi trường: Google Cloud Shell, Python 3.12.3, SQLite 3.45.1 (thư viện chuẩn sqlite3, cơ sở dữ liệu dạng tệp); không dùng dịch vụ tính phí.
- Phần bắt buộc S01–S15: sql/answers.sql trên data/sales.db (dữ liệu mẫu 5/5/6/8). Phần bổ sung S16–S21: sql/answers_homework.sql trên data/sales_extended.db.
- Kịch bản hỗ trợ: build_db.py, generate_data.py, run_sql.py, sql_console.py.

2. Tiêu chí 1 — Cài đặt và tạo dữ liệu (2,0 điểm)

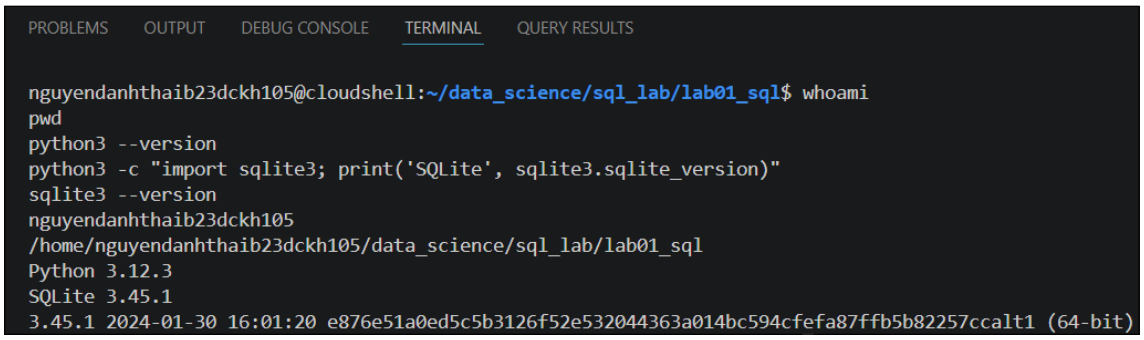
Lược đồ được định nghĩa trong sql/schema.sql với bốn bảng, khoá chính, khoá ngoại và ràng buộc miền giá trị. Khoá chính của order_items là tổ hợp (order_id, product_id) nên mỗi sản phẩm chỉ xuất hiện một lần trong một đơn. Mọi kết nối đều bật PRAGMA foreign_keys = ON vì SQLite không kiểm tra khoá ngoại mặc định.

```
CREATE TABLE customers (customer_id INTEGER PRIMARY KEY, name TEXT NOT NULL, city TEXT);
CREATE TABLE products  (product_id INTEGER PRIMARY KEY, name TEXT NOT NULL,
                        category TEXT, price REAL CHECK(price >= 0));
CREATE TABLE orders    (order_id INTEGER PRIMARY KEY, customer_id INTEGER NOT NULL,
                        order_date TEXT,
                        FOREIGN KEY(customer_id) REFERENCES customers(customer_id));
CREATE TABLE order_items (order_id INTEGER, product_id INTEGER,
                        quantity INTEGER CHECK(quantity > 0),
                        PRIMARY KEY(order_id, product_id),
                        FOREIGN KEY(order_id)   REFERENCES orders(order_id),
                        FOREIGN KEY(product_id) REFERENCES products(product_id));
```

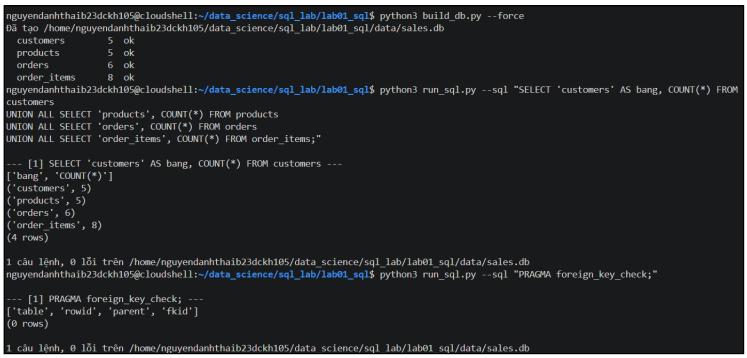
Bảng 1. Số dòng sau khi nạp dữ liệu mẫu

Bảng	Kỳ vọng của đề	Thực tế	Kết quả
customers	5	5	khớp
products	5	5	khớp
orders	6	6	khớp
order_items	8	8	khớp

Sau khi nạp, PRAGMA foreign_key_check trả về rỗng, nghĩa là mọi khoá ngoại đều trỏ tới bản ghi tồn tại. Kịch bản build_db.py in trực tiếp bảng đối chiếu trên nên việc kiểm tra không phụ thuộc vào thao tác thủ công; ảnh chụp màn hình tương ứng nằm ở mục 7.



Hình 1. Môi trường thực hành: tài khoản, thư mục, Python và SQLite



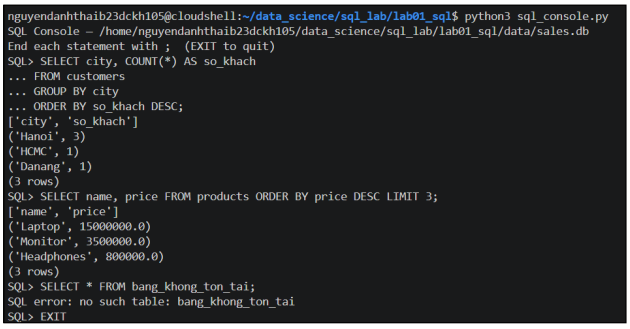
Hình 2. Tạo bốn bảng: số dòng 5/5/6/8 và foreign_key_check rỗng

3. Tiêu chí 2 — CRUD và câu cơ bản (2,0 điểm)

Nhóm S01–S07 gồm ba truy vấn đọc và bốn thao tác có ghi; các thao tác ghi được viết để tệp đáp án chạy lặp an toàn.

Bảng 2. Các câu S01–S07 và kết quả thực tế

Mã	Nội dung	Cách thực hiện	Kết quả
S01	Tên và thành phố khách hàng	SELECT, ORDER BY	5 dòng
S02	Sản phẩm giá trên 1 triệu	WHERE price > 1000000	2 dòng
S03	Sắp xếp sản phẩm theo giá giảm dần	ORDER BY DESC	5 dòng
S04	Thêm khách hàng mới (Lan, Hue)	INSERT … ON CONFLICT	1 dòng
S05	Tăng giá Mouse thêm 10%	UPDATE có điều kiện	1 dòng
S06	Xoá khách vừa thêm	DELETE + NOT EXISTS	1 dòng
S07	Mã đơn và tên khách hàng	INNER JOIN	6 dòng



Hình 3. sql_console.py: câu nhiều dòng, GROUP BY, một lỗi SQL, EXIT

- S04 dùng INSERT … ON CONFLICT(customer_id) DO NOTHING; S05 dùng UPDATE … WHERE product_id = 102 AND price = 250000 nên lần chạy thứ hai không khớp dòng nào (Rows affected: 0) và giá giữ 275.000 thay vì nhân 10% lần nữa.
- S06 dùng DELETE … AND NOT EXISTS (SELECT 1 FROM orders …) — đúng nghĩa “khách vừa thêm chưa có đơn”, không vi phạm khoá ngoại. Sau

hai lần chạy vẫn 5 khách hàng.

4. Tiêu chí 3 — Truy vấn nâng cao (3,0 điểm)

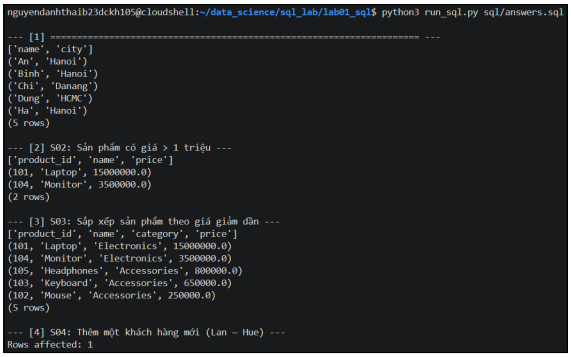
S08 nối ba bảng tính tổng tiền từng đơn; S09 gom theo khách hàng; S10 lọc kết quả tổng hợp bằng HAVING; S11 dùng truy vấn con vô hướng; S12 đếm số đơn; S13 lấy top 3 theo doanh thu.

```
SELECT o.order_id, SUM(i.quantity * p.price) AS total
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
JOIN products p    ON p.product_id = i.product_id
GROUP BY o.order_id
ORDER BY total DESC;
```

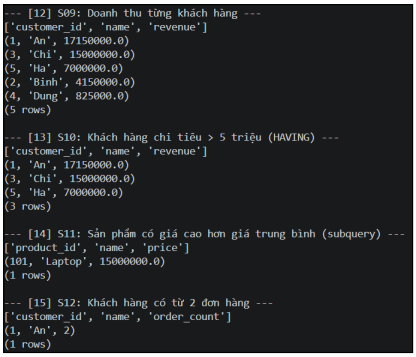
Bảng 3. Kết quả S08 (hai cột đầu) và S09 (hai cột sau)

Mã đơn	Tổng tiền (đồng)	Khách hàng	Doanh thu (đồng)
1001	15.550.000	An (id 1)	17.150.000
1004	15.000.000	Chi (id 3)	15.000.000
1006	7.000.000	Ha (id 5)	7.000.000
1002	4.150.000	Binh (id 2)	4.150.000
1003	1.600.000	Dung (id 4)	825.000
1005	825.000	—	—
Tổng	44.125.000	—	—

- Kiểm tra chéo đơn 1001: 1 Laptop 15.000.000 + 2 Mouse 275.000 = 15.550.000, khớp kết quả máy (giá 275.000 vì S05 chạy trước S08).
- S10 trả về 3 khách vượt 5 triệu (An, Chi, Ha); S11 chỉ Laptop vượt giá trung bình 4.045.000; S12 đúng 1 khách (An, 2 đơn); S13 top 3: Laptop 30.000.000, Monitor 10.500.000, Headphones 1.600.000.



Hình 4. Chạy sql/answers.sql: kết quả S01–S04



Hình 5. S09–S12: gom nhóm, HAVING, truy vấn con, đếm đơn

5. Tiêu chí 4 — Index và giải thích (1,5 điểm)

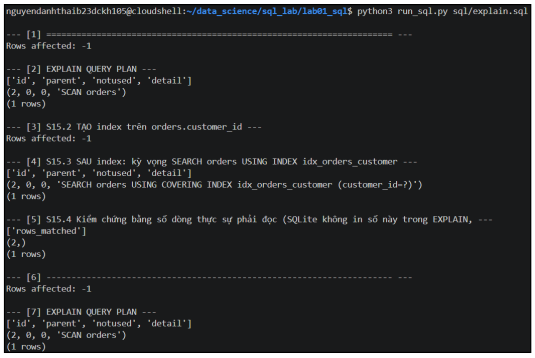
S15 cần so sánh kế hoạch trước và sau khi có chỉ mục, nên sql/explain.sql tự xoá chỉ mục, chạy EXPLAIN, tạo chỉ mục rồi chạy EXPLAIN lần hai (S20 lặp lại trên dữ liệu lớn).

```
DROP INDEX IF EXISTS idx_orders_customer;
EXPLAIN QUERY PLAN SELECT order_id FROM orders WHERE customer_id = 1;
CREATE INDEX IF NOT EXISTS idx_orders_customer ON orders(customer_id);
EXPLAIN QUERY PLAN SELECT order_id FROM orders WHERE customer_id = 1;
```

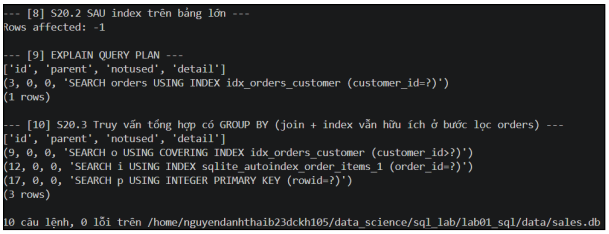
Bảng 4. Kế hoạch thực thi trước và sau khi tạo chỉ mục

Thời điểm	Kế hoạch thực thi
Trước khi tạo chỉ mục	SCAN orders
Sau khi tạo chỉ mục	SEARCH orders USING COVERING INDEX idx_orders_customer (customer_id=?)

- SCAN đọc tuần tự toàn bảng rồi lọc nên chi phí tăng theo số dòng; SEARCH đi xuống cây B-tree của chỉ mục nên chi phí tỉ lệ với logarit số dòng cộng số dòng khớp.
- Kế hoạch ghi COVERING INDEX vì truy vấn chỉ cần order_id — với khoá chính INTEGER, chỉ mục trên customer_id đã chứa sẵn rowid nên không phải đọc lại bảng. Với 6 dòng, chênh lệch thời gian dưới ngưỡng đo, nên kết luận dựa trên kế hoạch thực thi.



Hình 6. sales.db: SCAN orders trước và sau index



Hình 7. S20.2–S20.3: SEARCH … USING INDEX trên dữ liệu lớn

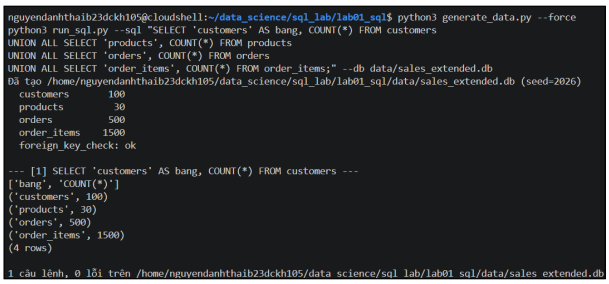
6. Tiêu chí 5 — Bài mở rộng và báo cáo (1,5 điểm)

Dữ liệu mở rộng sinh bằng generate_data.py với random.seed(2026) nên tái lập được, ghi vào tệp riêng data/sales_extended.db, không ghi đè dữ liệu mẫu.

Bảng 5. Quy mô dữ liệu mở rộng so với yêu cầu của đề

Đối tượng	Yêu cầu tối thiểu	Thực tế
Khách hàng	100	100
Sản phẩm	30	30
Đơn hàng	500	500
Dòng đơn hàng	1.000	1.500

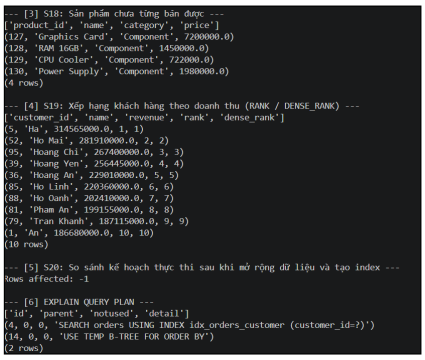
- S16 doanh thu theo tháng bằng strftime('%Y-%m', order_date): 13 nhóm từ 10/2025 đến 10/2026, cao nhất 2025-10 với 1.142.485.000 đồng; tháng cuối chỉ 7 đơn vì dữ liệu kết thúc giữa tháng.
- S17 khách chưa có đơn (LEFT JOIN … IS NULL): 8 khách. S18 sản phẩm chưa từng bán (NOT EXISTS): 4 sản phẩm.
- S19 xếp hạng bằng RANK và DENSE_RANK; dữ liệu không có đồng hạng nên hai cột trùng nhau, khi có đồng hạng RANK cho 2, 2, 4 còn DENSE_RANK cho 2, 2, 3.
- S20 trên dữ liệu lớn: SEARCH orders USING INDEX kèm USE TEMP B-TREE FOR ORDER BY do sắp xếp theo order_date. S21: SQL Server có #t (session), ##t (toàn cục) và tempdb; SQLite chỉ có TEMP TABLE gắn với kết nối.



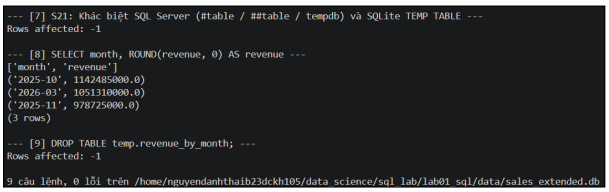
Hình 8. Sinh dữ liệu mở rộng: 100/30/500/1500 dòng



Hình 9. S16 doanh thu 13 tháng và S17 khách chưa có đơn



Hình 10. S18 bốn sản phẩm chưa bán, S19 RANK/DENSE_RANK, S20



Hình 11. S21 TEMP TABLE và truy vấn tổng hợp trên bảng tạm

7. Danh mục ảnh minh chứng

Toàn bộ ảnh dưới đây được chụp trực tiếp trên Google Cloud Shell và đã chèn vào đúng mục 2–6 của báo cáo; bảng này để đối chiếu tệp với nội dung minh chứng.

Bảng 6. Các ảnh minh chứng kèm theo bài nộp

Tệp trong evidence/	Nội dung cần thấy	Mục
01_env.png	whoami, pwd, Python 3.12.3, SQLite 3.45.1	2
02_tables_count.png	Số dòng 5/5/6/8, foreign_key_check rỗng	2
03_console.png	Console: câu nhiều dòng, một lỗi SQL, EXIT	3
04_s08_result_01, _02	Kết quả S01–S04 và S09–S12	4
05_explain_before_after_01, _02	SCAN orders → SEARCH … USING INDEX	5
06_expanded_counts.png	100/30/500/1500 dòng, khoá ngoại hợp lệ	6
07_s19_rank_01, _02, _03	S16, S17, S18, S19, S20, S21	6

8. Kết luận

Đã hoàn thành đủ 21 yêu cầu của đề: lược đồ bốn bảng có ràng buộc, dữ liệu mẫu đúng 5/5/6/8 dòng, truy vấn từ cơ bản đến nâng cao, chỉ mục có phân tích kế hoạch thực thi; các tệp đáp án chạy lặp an toàn và dữ liệu mở rộng tái lập được nhờ hạt giống cố định. Hai điểm rút ra: SQLite phải bật PRAGMA foreign_keys = ON ở mọi kết nối, và đánh giá chỉ mục nên dựa trên kế hoạch thực thi thay vì thời gian chạy khi dữ liệu còn nhỏ.